ADITYA UNIVERSITY

SCHOOL OF ENGINEERING

Department of Artificial Intelligence and Machine Learning

CERTIFICATE

This is to certify that the Cornerstone project work entitled *"***EMPLOYEE RECORD MANAGEMENT SYSTEM (MERN CRUD)***"* is submitted by

A TEJASRI SAI SANTHOSH	**(24B11AI019)**
B KALYANI	**(24B11AI044)**
G MOUNIKA	**(24B11AI135)**
J AKSHARA HASINI	**(24B11AI146)**

in partial fulfillment of the requirements for the award of the B.Tech. degree in Artificial Intelligence and Machine Learning for the academic year 2025-2026.

Project Guide

Mrs. P. Venkata Bala Annapurna, M.Tech(PhD),
Assistant Professor,
Department of AIML

Head of the Department

Dr. M. Venkatesh, M.Tech., Ph.D,
Associate Professor,
Department of AIML

DECLARATION

We hereby declare that the Cornerstone project entitled "**EMPLOYEE RECORD MANAGEMENT SYSTEM (MERN CRUD)**" is original and is submitted to **ADITYA UNIVERSITY**, Surampalem, in partial fulfillment of the requirements for the award of the **B.Tech. degree** in **Artificial Intelligence and Machine Learning**. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy

Place: Surampalem

Date:

By

A T SAI SANTHOSH	(24B11AI019)
B KALYANI	(24B11AI044)
G MOUNIKA	(24B11AI135)
J AKSHARA HASINI	(24B11AI146)

ACKNOWLEDGEMENT

It is with immense pleasure that we would like to express us indebted gratitude to our project Guide **Mrs. P. Venkata Bala Annapurna**, who has guided us a lot and encourage in every step of the project work, her valuable moral support and guidance throughout the project helped us a greater extent.

We are grateful to **Dr. Maganti Venkatesh, Assoc. Prof.** and **HoD,** for inspiring us all the way and for arranging all the facilities and resources needed for our project.

We wish to thank **Dr. M.V. Rajesh, Assoc. Dean SoE.** for the encouragement and support during the course of my project.

We would like to extend our sincere thanks to **Dr. A. Ramesh, Pro Vice-Chancellor (Engg. & Sci.), Dr. G. Suresh, Registrar, Dr. S.Rama Sree, Pro Vice-Chancellor (Academics), Dr. M.B. Srinivas, Vice-Chancellor, Dr. M. Sreenivasa Reddy, Deputy Pro-Chancellor and Management, Aditya University** for unconditional support for providing us the best infrastructural facilities and state of the art laboratories during our project.

Not to forget, **Parents Faculty, Teaching, Non-Teaching Staff and our Friends** who have directly or indirectly supported us in completing this project on time.

ABSTRACT

The Employee Record Management System is designed to streamline and digitize the process of managing employee information within an organization. In many institutions, employee data is often handled manually or through basic tools, leading to inefficiencies such as data redundancy, difficulty in retrieving records, and lack of proper organization. This project addresses these challenges by providing a centralized web-based application developed using the MERN stack (MongoDB, Express.js, React.js, and Node.js).

The system focuses on implementing core CRUD (Create, Read, Update, Delete) operations to efficiently manage employee data such as name, email, department, and salary. It enables administrators to easily add new employee records, update existing information, delete outdated records, and view all employee details in a structured manner.

The frontend of the application is built using React.js, providing a responsive and user-friendly interface, while the backend is developed using Node.js and Express.js to handle server-side logic and RESTful API communication. MongoDB is used as the database to ensure efficient storage and retrieval of data. As an enhancement, the system also includes a simple login feature with role-based access, where administrators can manage all records and employees can view their own details.

Keywords:

Employee Management, MERN Stack , CRUD Operations , Data Management , Web Application.

LIST OF CONTENTS

CONTENTS	PAGE NO
LIST OF FIGURES	i
LIST OF TABLES	ii
Chapter 1: INTRODUCTION	1
1.1 Introduction of Project Domain	1
1.2 Problem Statement	1
1.3 Objectives of the Project	1
1.4 Scope of the Project	1
Chapter 2: SYSTEM OVERVIEW	2
2.1 Existing System	2
2.2 Limitations of Existing System	2
2.3 Proposed System	2
2.4 Software Requirements	2
2.5 Hardware Requirements	2
Chapter 3: TECHNOLOGIES USED	3
3.1 React.js	3
3.2 Node.js	3
3.3 Express.js	3
3.4 MongoDB	3
3.5 Mongoose	3
Chapter 4: DATABASE DESIGN	4
4.1 Database Table	4
4.2 Table Structure	4
4.3 Keys and Constraints	5
4.4 Table Relationships	5
4.5 Sample Data / Sample Record	5

Chapter 5: SYSTEM MODULES 6

Chapter 6: IMPLEMENTATION 7

6.1 Input Validation 7

6.2 Exception Handling 7

6.3 Data Processing and Transformation 7

6.4 CRUD Implementation 7

6.5 Result Generation 8

Chapter 7: OUTPUT SCREENS 9

7.1 Login Page 9

7.2 Admin Dashboard 10

7.3 Add Employee Form 11

7.4 Employee Records Display 12

7.5 Update Employee Details 13

7.6 Delete Employee Details 14

7.7 Search Employee Details 16

7.8 Filter Employee Data 17

Chapter 8: CONCLUSION 18

Chapter 9: FUTURE ENHANCEMENTS 19

Chapter 10: REFERENCES 20

APPENDIX 21

A. Sample Code 21

B. Screenshots and System outputs 25

C. API Endpoints 27

D. Project Structure 28

E. Installation and Execution Steps 30

LIST OF FIGURES

FIGURE NO.	NAME OF THE FIGURE	PAGE NO
7.1	Login Page	9
7.2	Admin Dashboard	10
7.3	Add Employee Form	11
7.4	Employee Records Display	12
7.5	Update Employee Details	13
7.6.1	Employee Data Before Deletion	14
7.6.2	Pop-Up Message During Deletion	14
7.6.3	Employee Data After Deletion	15
7.7	Search Employee by Name	16
7.8	Filter by Department and Status	17
11.1.1	MongoDB Database View	25
11.2.1	Employee Records in MongoDB	26
11.3.1	Backend Server Running with MongoDB Connection	26

LIST OF TABLES

TABLE NO.	NAME OF THE TABLE	PAGE NO
4.1	Employee Data	4
4.2	Sample Employee Data	5

1. INTRODUCTION

1.1 Introduction

Employee data management plays a crucial role in organizations for maintaining accurate and organized records. This project focuses on developing a web-based Employee Record Management System using the MERN stack to efficiently manage employee details. The system helps in storing, updating, and retrieving employee information in a structured manner, improving overall efficiency and reducing manual effort.

1.2 Problem Statement

In traditional systems, employee records are often maintained manually or using basic tools like spreadsheets, which are time-consuming and prone to errors. It becomes difficult to manage, update, and retrieve data efficiently. Therefore, there is a need for an automated system that can handle employee data effectively with better accuracy, faster access, and improved organization.

1.3 Objectives

- To develop a web-based Employee Record Management System
- To implement CRUD operations for managing employee data
- To enable search functionality for quick data retrieval
- To provide filtering options based on department
- To create a user-friendly and responsive interface

1.4 Scope

The system can be used in organizations to manage employee records efficiently. It helps administrators maintain accurate data, perform updates easily, and retrieve information quickly using search and filtering features. The system improves productivity and reduces manual errors in managing employee information.

2. SYSTEM OVERVIEW

2.1 Existing System

Currently, employee data is managed manually or using basic tools like spreadsheets. There is no centralized system to efficiently store, update, and retrieve employee information.

2.2 Limitations of Existing System

- Time-consuming process
- Prone to human errors
- Difficult to manage large employee data
- No proper search and filtering system

2.3 Proposed System

The proposed system is a web-based Employee Record Management System developed using the MERN stack. It provides a centralized platform to manage employee data with CRUD operations. The system includes search and filtering features to quickly retrieve records and improves overall efficiency.

2.4 Software Requirements

- Node.js
- MongoDB
- React.js
- Express.js
- Web Browser (Chrome/Edge)
- Code Editor (VS Code)

2.5 Hardware Requirements

A basic computer system with at least 4GB RAM and standard processor is sufficient. No high-end hardware is required. It can run on any normal desktop or laptop.

3. TECHNOLOGIES USED

3.1 React.js

React.js is a popular JavaScript library used for building user interfaces, especially for single-page applications. In this project, React.js is used to develop the frontend of the Employee Record Management System. It enables the creation of reusable components such as forms, tables, and dashboards, making the UI more organized and maintainable.

3.2 Node.js

Node.js is used as the runtime environment to execute backend code and handle server-side operations in the application.

3.3 Express.js

Express.js is a lightweight web framework for Node.js that simplifies the development of backend applications. It is used in this project to create RESTful APIs for performing CRUD operations on employee data. Express.js manages routing, middleware, and request handling efficiently.

3.4 MongoDB

MongoDB is a NoSQL database used to store and manage application data in a flexible, document-oriented format. In this project, MongoDB is used to store employee records such as name, email, department, and salary. Unlike traditional relational databases, MongoDB stores data in JSON-like documents, which makes it easier to handle dynamic and unstructured data.

3.5 Mongoose

Mongoose is an Object Data Modeling (ODM) library for MongoDB and Node.js. It is used in this project to define schemas and models for the database. Mongoose provides a structured way to interact with MongoDB by enforcing data validation, defining data types, and managing relationships between data.

4. DATABASE DESIGN

4.1 Database Table

Employee Table

The system uses a MongoDB database named **employee_db** to store employee-related information. The main collection used in the system is the **Employee Collection**, which stores all details of employees such as name, email, department, and salary.

4.2 Table Structure

Column Name	Data Type	Description
_id	ObjectId	Unique identifier for each employee (auto-generated)
name	String	Name of the employee
email	String	Email address of the employee
department	String	Department in which the employee works
salary	Number	Salary of the employee
username	String	Unique login username for each employee
password	String	Encrypted (hashed) password for secure authentication
position	String	Designation or job role of the employee
status	String	Employee Status (Active/Inactive)

Table 4.1: Employee Table

4.3 **Keys and Constraints**

4.3.1 Primary Key

- _id → Automatically generated unique identifier for each record in MongoDB

4.3.2 Constraints

- NOT NULL → name, email, department, salary
- Email must be unique for each employee
- Salary must be a positive numeric value
- Department should contain valid department names

4.4 Table relationships:

No relationships are maintained as the system uses a single collection structure. All employee data is stored in one collection, making it simple and efficient for CRUD operations.

4.5 Sample Data / Sample Record:

_id	name	Email	department	salary
001	Alice	alice@gmail.com	IT	30000
002	Bob	bob@gmail.com	HR	28000
003	Charlie	charlie@gmail.com	Finance	32000
004	David	david@gmail.com	Engineering	15000
005	Eulean	eluean@gmail.com	Arts	27000

Table 4.2: *Sample Employee* ***Data***

5. SYSTEM MODULES

Module 1: Employee Management (CRUD Operations)

This module is responsible for managing employee records in the system. It allows the administrator to add new employees, view existing records, update employee details such as department and salary, and delete employee records when necessary.

Module 2: Data Storage (Database Management)

This module handles the storage and retrieval of employee data using MongoDB. All employee details are stored in a centralized database, ensuring data persistence and reliability.

Module 3: Search Functionality

This module provides the ability to search employee records based on specific criteria such as employee name. It helps users quickly locate required information without manually browsing through all records, thereby improving efficiency and user experience.

Module 4: Filter System

This module allows filtering of employee records based on department. It helps in organizing and displaying data in a structured way, making it easier to analyze employees belonging to specific departments.

Module 5: User Interface (Frontend)

This module is responsible for providing an interactive and user-friendly interface using React.js. It includes forms for adding and updating employee details, tables for displaying records, and UI elements for search and filtering.

Module 6: Authentication (Enhancement)

This module provides a simple login system for users. It allows administrators and employees to access the system securely. Role-based access is implemented where administrators can manage all employee data, while employees can view their own details. This module enhances security and usability of the system.

6. IMPLEMENTATION

6.1 Input Validation

Before storing employee data in the system, input validation is performed to ensure correctness and consistency. Fields such as name, email, department, and salary are checked for valid values. Empty or invalid inputs are not accepted, and proper formats (such as valid email structure and numeric salary values) are enforced.

6.2 Exception Handling

The system handles errors that may occur during data processing and user interaction. Common issues such as invalid input, server errors, or database connection failures are managed effectively. Proper error messages are displayed to the user, ensuring smooth operation of the application without crashes.

6.3 Data Processing and Transformation

The frontend sends user input data to the backend through API requests. The backend processes the data and performs necessary operations before storing it in MongoDB. Data is structured according to the defined schema using Mongoose, ensuring consistency and proper formatting of employee records.

6.4 CRUD Implementation

The system implements CRUD operations to manage employee data:

- **Create** → Adding new employee records
- **Read** → Viewing employee details
- **Update** → Modifying existing employee information
- **Delete** → Removing employee records

These operations are performed through RESTful APIs built using Express.js.

6.5 Result Generation

The system displays employee data in a structured format on the user interface. The results include a list of employees along with their details such as name, email, department, and salary. Search and filtering features dynamically update the displayed data, allowing users to quickly find relevant information.

7. OUTPUT SCREENS

The following section presents the output screens of the system. These screenshots demonstrate the various functionalities of the Employee Record Management System, including employee data management, CRUD operations, and search and filtering features.

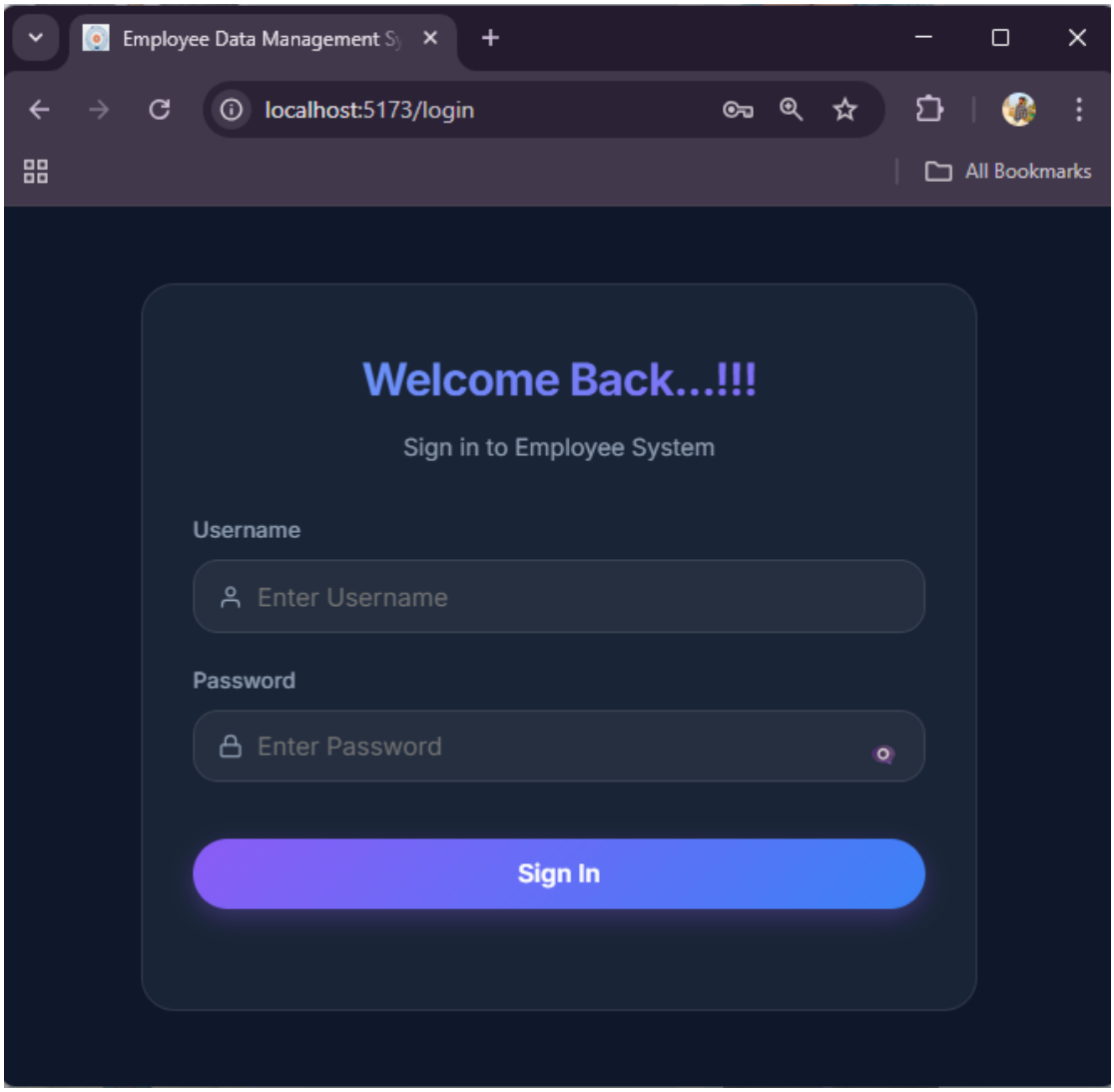

Figure 7.1: *Login Page*

Login Page:

The Login Page allows users to securely access the system by entering their username and password. It verifies the credentials and redirects the user to the dashboard upon successful login. It also includes a password visibility toggle for better usability.

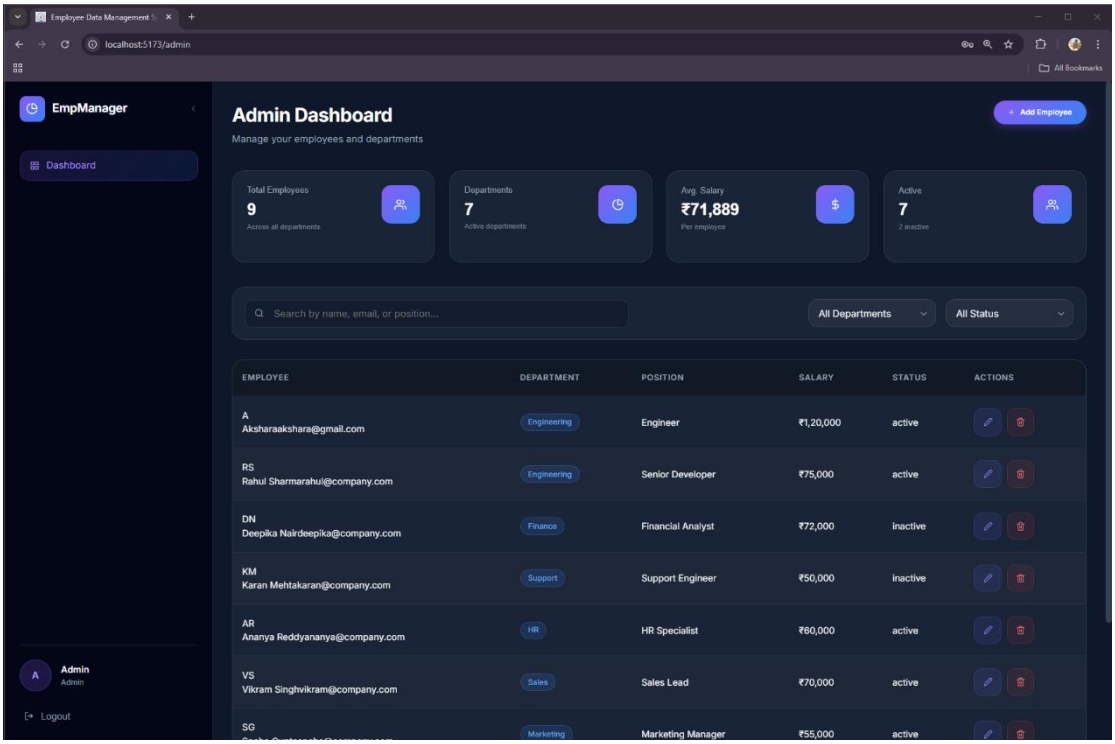

Figure 7.2: Admin Dashboard

Admin Dashboard:

The Admin Dashboard is the main interface where the administrator can manage employee records. It provides features such as viewing, adding, updating, and deleting employees, along with search and filtering options for efficient data management.

From this interface, the administrator can perform all CRUD operations, including adding new employees, updating existing records, and deleting employee data. The dashboard also includes a search bar to quickly find employees by name and filtering options to view employees based on department.

The user interface is designed to be simple and interactive, ensuring easy navigation and efficient management of employee data. It allows administrators to perform operations smoothly without needing technical expertise.

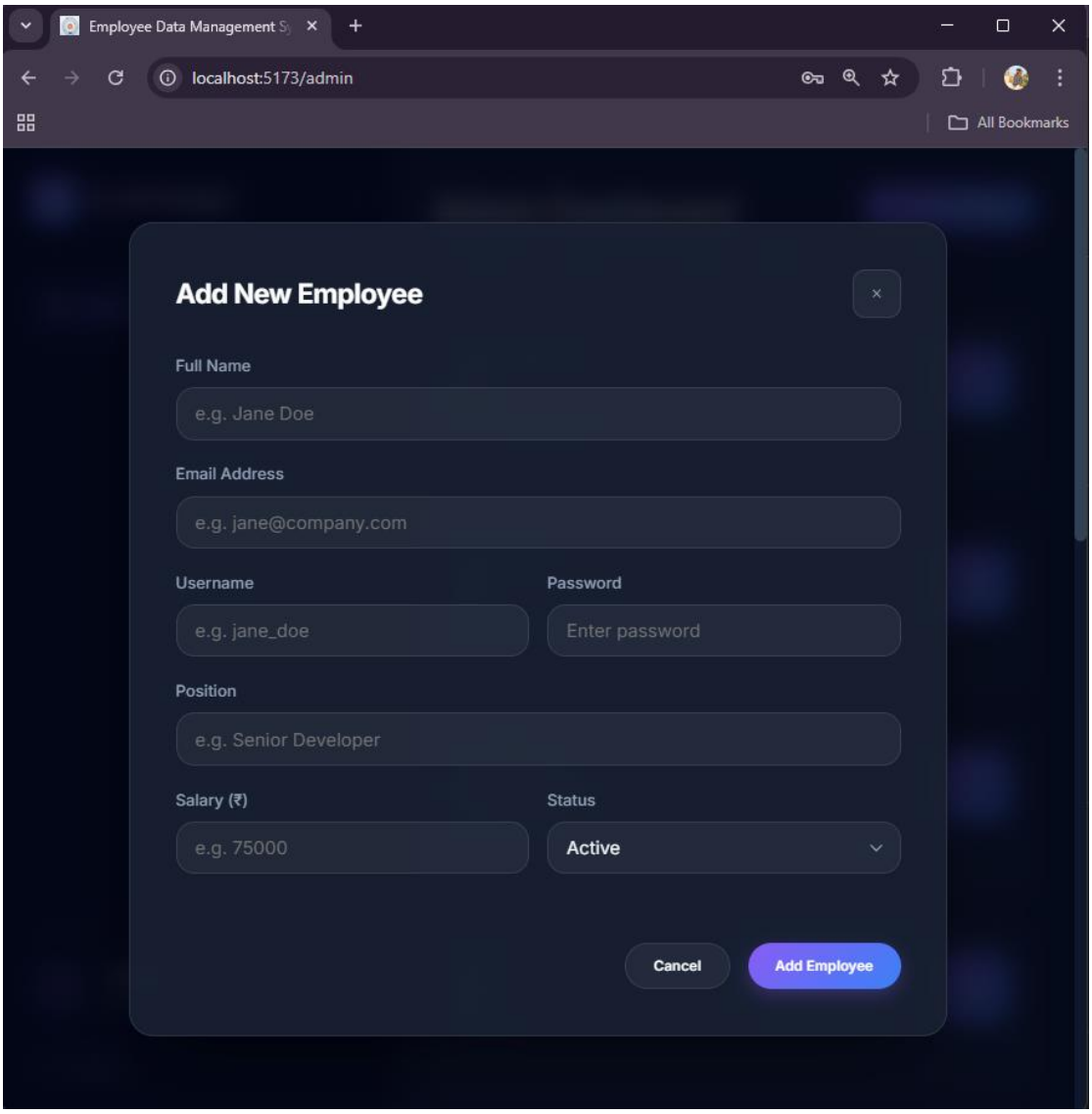

Figure 7.3: Add Employee Form

Add Employee Form:

The Add Employee Form allows the administrator to enter and store new employee details such as name, email, department, and salary. It validates the input and updates the employee list after successful submission.

The form is designed with a simple and user-friendly interface, making it easy for administrators to add new records efficiently.

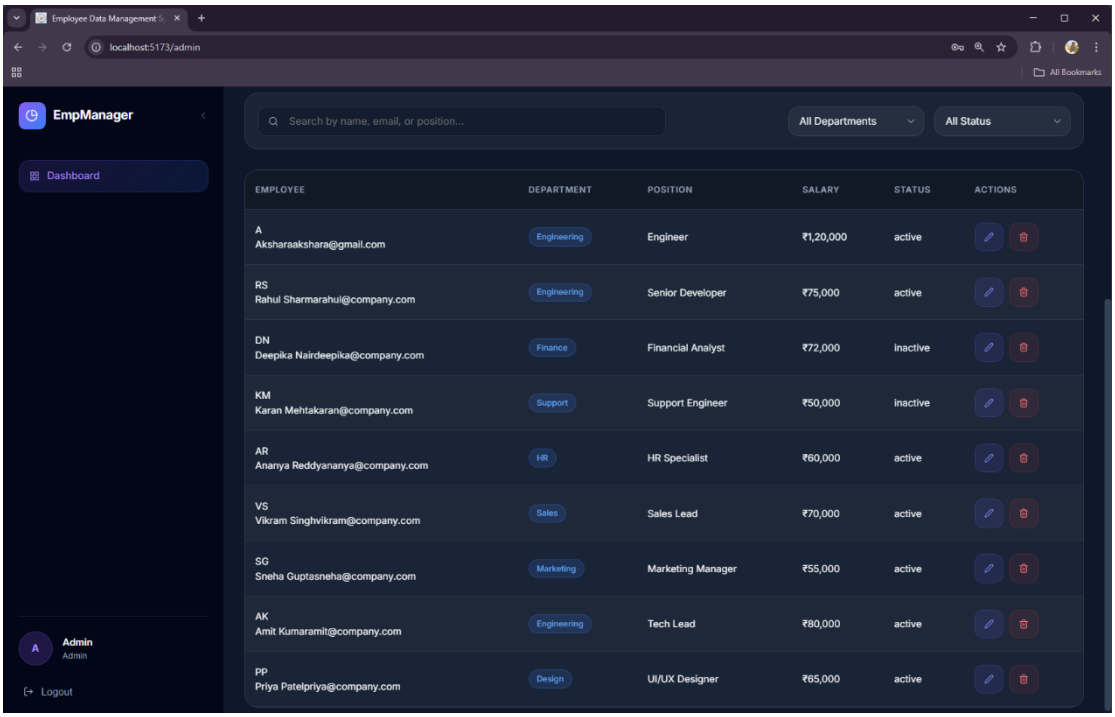

Figure 7.4: *Employee Records Display*

Employee Records:

The Employee Records section displays all the employee data stored in the system in a structured table format. It includes details such as name, email, department, and salary for each employee. This section provides a clear and organized view of all records, making it easy for the administrator to manage employee information.

The table also includes options to update and delete employee records, allowing the administrator to modify existing data or remove records when necessary. Additionally, search and filtering features are integrated to quickly locate specific employees based on name or department.

This section ensures efficient data management and easy accessibility of employee information.

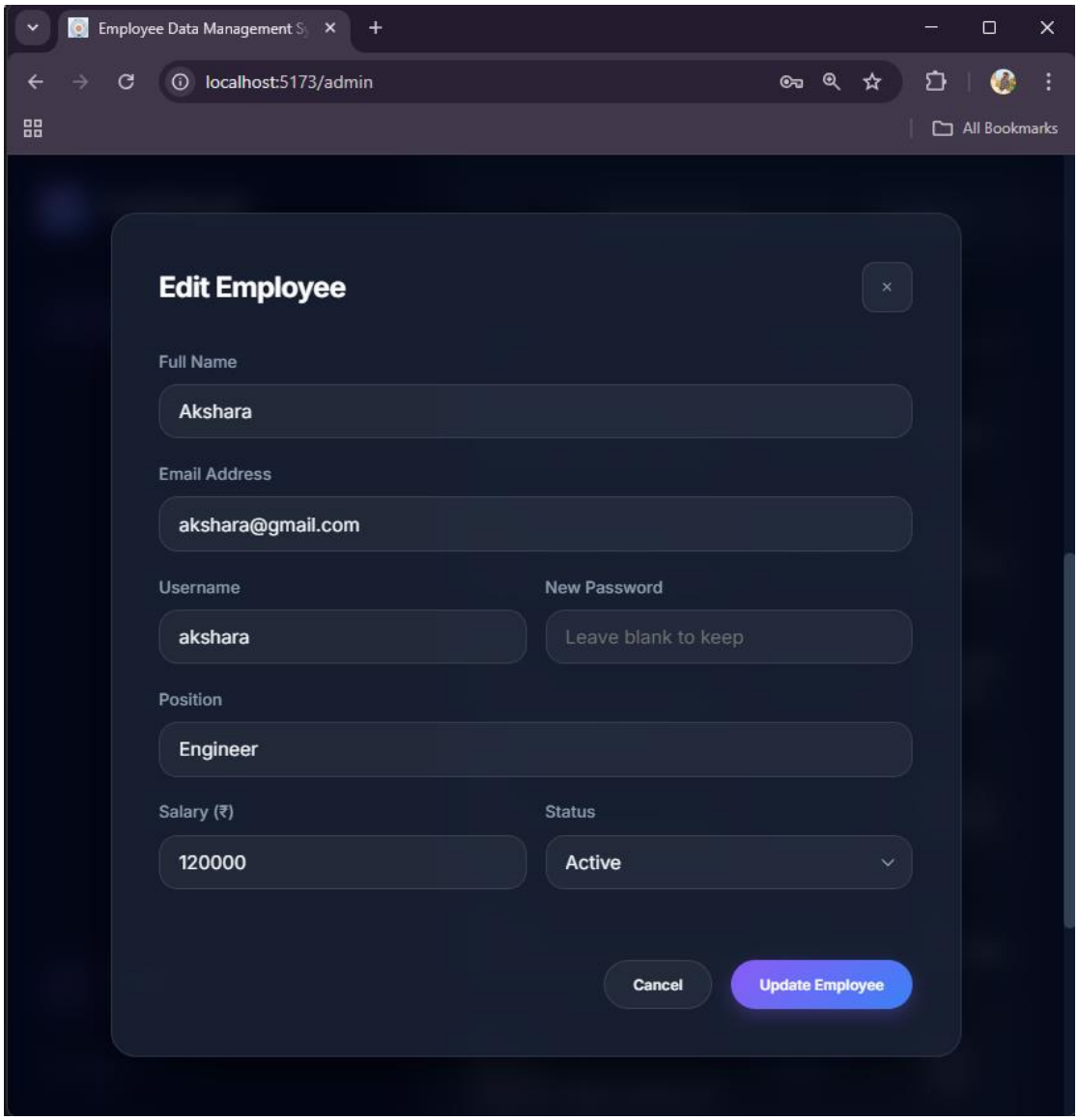

Figure 7.5: *Update Employee Details*

Update Employee:

The Update Employee feature allows the administrator to modify existing employee details in the system. By selecting an employee record from the list, the administrator can access a pre-filled form containing the current information such as name, email, department, and salary.This feature helps in maintaining accurate and up-to-date employee information within the system.

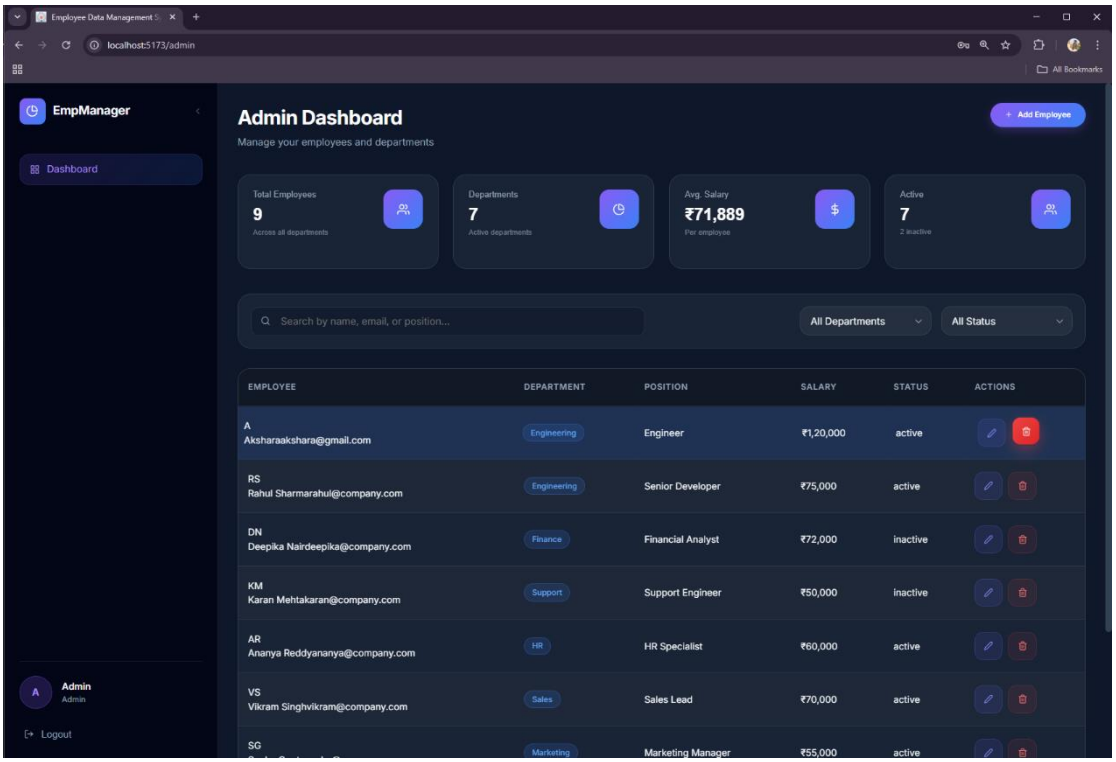

Figure 7.6.1: *Employee Data Before Deletion*

(Note: Deleting Rahul Sharma's Employee Details)

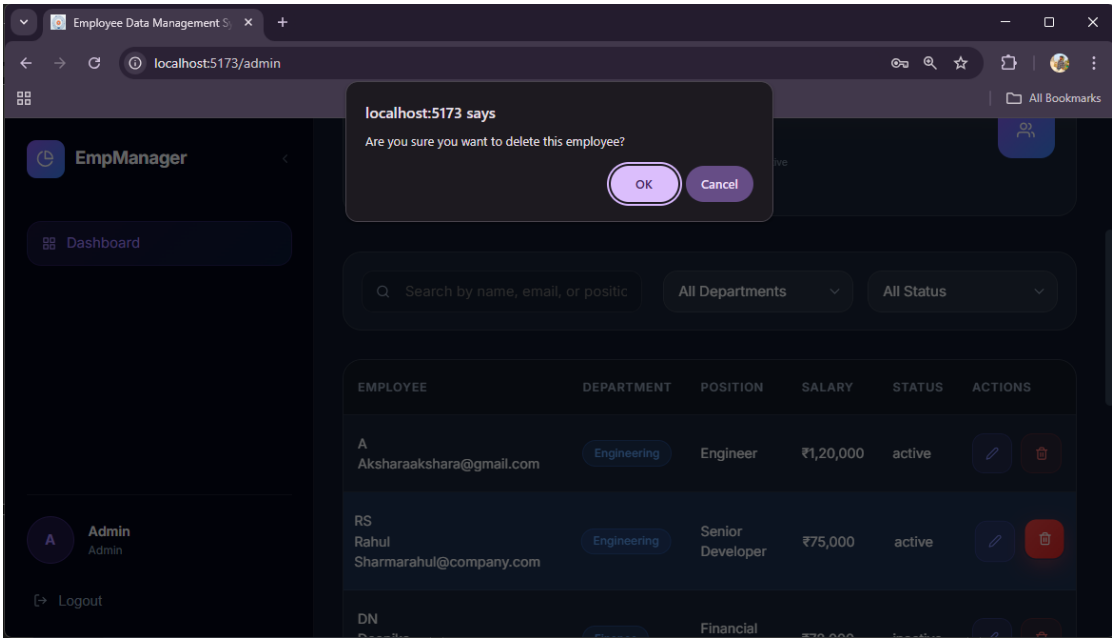

Figure 7.6.2: *Pop-Up Message during Deletion*

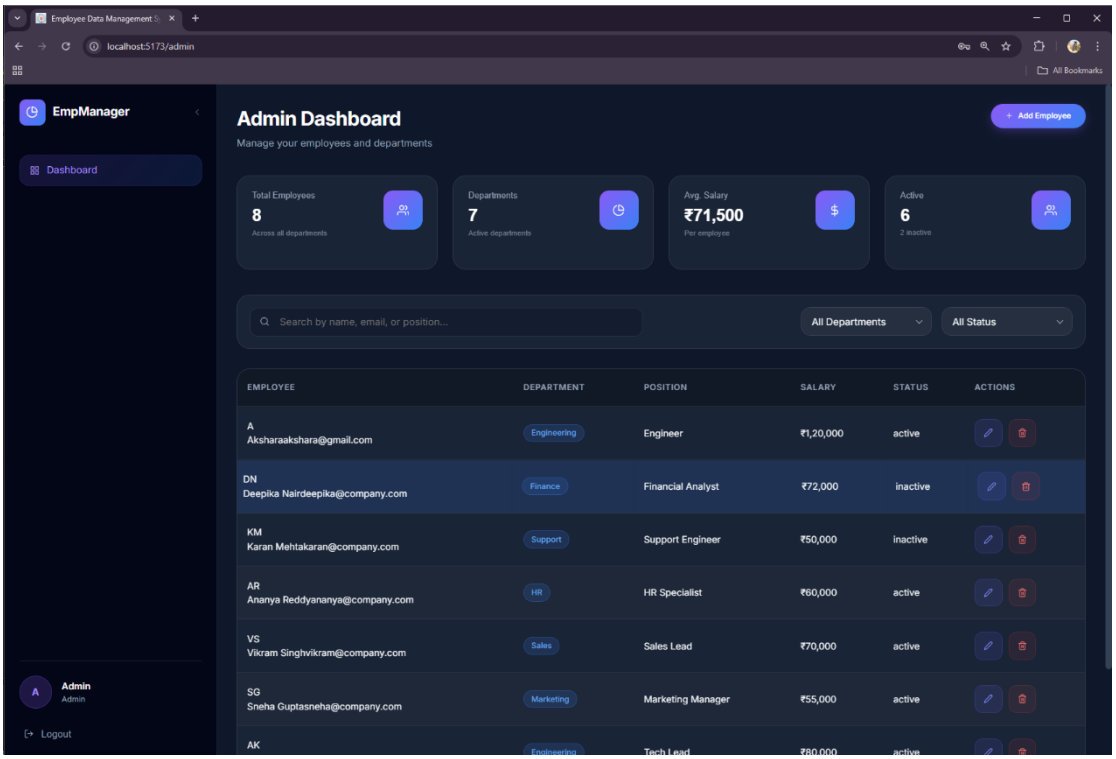

Figure 7.6.3: *Employee Data After Deletion*

(Note: Deleted Rahul Sharma's Employee Details)

Delete Employee:

The Delete Employee feature allows the administrator to remove employee records from the system when they are no longer required. Each employee entry in the records table includes a delete option, which enables quick and easy removal of specific records.

When the delete action is performed, the selected employee data is permanently removed from the database. The system ensures that the deletion process is handled smoothly, and the updated employee list is immediately reflected on the user interface.

This feature helps in maintaining accurate and up-to-date records by eliminating unnecessary or outdated employee data.

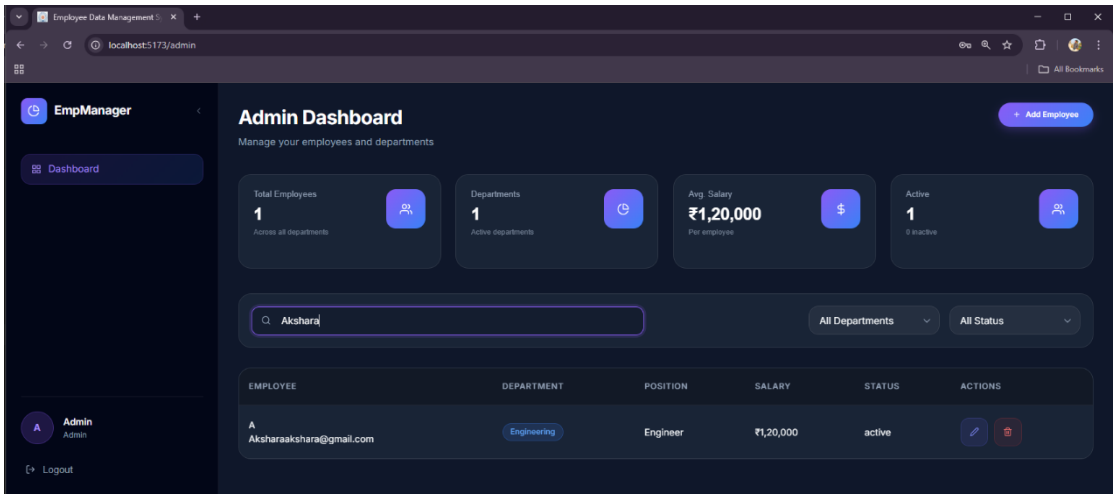

***Figure** 7.7: Search Employee by Name*

(Note: Akshara's Details has been Searched)

Search Functionality:

The Search Functionality allows the administrator to quickly find specific employee records based on the employee name. A search bar is provided in the dashboard where the user can enter a keyword, and the system dynamically filters the employee list to display matching results.

This feature improves efficiency by eliminating the need to manually browse through all records. It provides real-time results as the user types, making data retrieval faster and more convenient.

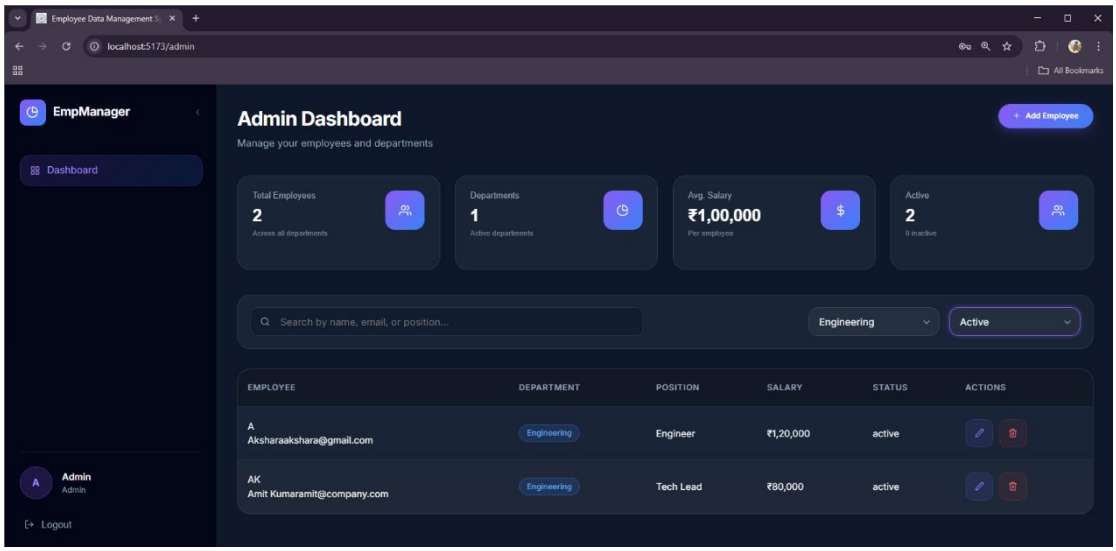

Figure 7.8: *Filter by Department and Status*

(Note: Employees in Engineering Department having Active Status been Filtered)

Filter Functionality:

The Filter Functionality allows the administrator to view employee records based on specific criteria, such as department. A filter option is provided in the dashboard, enabling users to select a particular department and display only the employees belonging to that category.

This feature helps in organizing and analyzing data more effectively by narrowing down the results. It improves usability by allowing quick access to grouped information without manually searching through all records.

8. CONCLUSION

The Employee Record Management System developed in this project successfully demonstrates the implementation of a full-stack web application using the MERN stack. The system provides an efficient and centralized platform for managing employee data, replacing traditional manual methods with a more organized and reliable digital solution.

The application effectively implements CRUD (Create, Read, Update, Delete) operations, allowing administrators to add, view, update, and delete employee records with ease. Additionally, the inclusion of search and filtering functionalities enhances data retrieval, making the system more efficient and user-friendly. The use of React for frontend, Node.js and Express.js for backend, and MongoDB for data storage ensures smooth communication between different components of the application.

As an enhancement, the system also incorporates a simple login mechanism with role-based access, improving usability and basic security. The project highlights the practical use of modern web technologies in developing scalable and responsive applications.

Overall, the system improves data organization, reduces manual effort, and provides a reliable solution for employee data management. The project can be further enhanced by adding advanced features such as data analytics, pagination, and improved security mechanisms.

9. Future Enhancements

- **Advanced Authentication System –** Implement secure authentication using JWT and role-based access control for better security and user management.
- **Pagination Support –** Add pagination to efficiently handle and display large numbers of employee records.
- **Profile Management –** Allow employees to update their own details such as contact information and profile data.
- **Data Visualization & Reports –** Integrate charts and dashboards to analyze employee data such as department distribution and salary trends.
- **Search Optimization –** Enhance search functionality with advanced filters such as salary range and multi-criteria search.
- **File Upload Feature –** Enable uploading of employee profile images or documents for better record management.
- **Responsive Design Improvement –** Optimize the application for mobile and tablet devices for better accessibility.
- **Performance Optimization –** Improve system performance by optimizing database queries and API responses.
- **Cloud Deployment –** Deploy the application on cloud platforms for better scalability and remote access.
- **Notification System –** Add alerts or notifications for important actions such as updates or record changes.

10. REFERENCES

1. **MongoDB Official Website – https://www.mongodb.com/**
   – For understanding database concepts and MongoDB usage
2. **React Official Documentation – https://react.dev/**
   – For learning frontend development using React.js
3. **Node.js Official Website – https://nodejs.org/**
   – For backend development and server-side programming
4. **Express.js Documentation – https://expressjs.com/**
   – For building RESTful APIs and handling routing
5. **Mongoose Documentation – https://mongoosejs.com/**
   – For schema design and MongoDB interaction
6. **GeeksforGeeks – https://www.geeksforgeeks.org/**
   – For tutorials on MERN stack and web development concepts
7. **MDN Web Docs – https://developer.mozilla.org/**
   – For HTML, CSS, and JavaScript concepts
8. **W3Schools – https://www.w3schools.com/**
   – For basic web development references and examples

APPENDIX

Appendix A: Sample Code

This section includes sample code snippets used in the development of the Employee Record Management System.

A.1 Employee Schema (Mongoose Model)

```
const mongoose = require('mongoose');

// Define the Employee schema with required fields
const employeeSchema = new mongoose.Schema({
    name: {
        type: String,
        required: true,
        trim: true
    },
    email: {
        type: String,
        required: true,
        unique: true,
        trim: true
    },
    username: {
        type: String,
        required: true,
        unique: true,
        trim: true
    },
    password: {
        type: String,
        required: true
    },
    department: {
```

```
            type: String,
            required: true,
            trim: true
        },
        position: {
            type: String,
            required: true,
            trim: true
        },
        salary: {
            type: Number,
            required: true,
            min: 0
        },
        status: {
            type: String,
            enum: ['active', 'inactive'],
            default: 'active'
        }
    }, {
        timestamps: true
    });

    module.exports = mongoose.model('Employee',
    employeeSchema);
```

A.2 Create Employee API

```
router.post('/', async (req, res) => {
const { name, email, username, password,
department, position, salary, status } = req.body;

const existingEmail = await Employee.findOne({ email });
if (existingEmail) {
     return res.status(400).json({ message: "Employee
with this email already exists" });
```

```
}

const hashedPassword = await bcrypt.hash(password, 10);

const newEmployee = new Employee({
        name,
        email,
        username,
        password: hashedPassword,
        department,
        position,
        salary,
        status
    });

    await newEmployee.save();
    res.status(201).json(newEmployee);
});
```

A.3 Get Employees API (Search + Filter)

```
router.get('/', async (req, res) => {
 try {
    const { search, department } = req.query;
     let query = {};

      if (search) {
                query.name = { $regex: search,
    $options: 'i'                 };
        }

        if (department && department !== 'All') {
            query.department = department;
        }
```

```
const employees = await Employee.find(query).sort({
createdAt: -1 });

        res.status(200).json(employees);

    } catch (error) {

res.status(500).json({ message: "Failed to fetch
employees" });

    }

});
```

A.4 Update Employee API

```
 router.put('/:id', async (req, res) => {

  const employee = await
Employee.findById(req.params.id);

employee.name = req.body.name || employee.name;

employee.email = req.body.email || employee.email;

employee.department = req.body.department ||
employee.department;

employee.salary = req.body.salary || employee.salary;


 await employee.save();

res.json(employee);

 });
```

A.5 Delete Employee API

```
router.delete('/:id', async (req, res) => {

 await Employee.findByIdAndDelete(req.params.id);

 res.json({ message: "Employee deleted successfully" });

 });
```

A.6 Authentication Middleware

```
 const isAuthenticated = (req, res, next) => {

  if (req.session && req.session.userId) {

     return next();
```

```
}

        return res.status(401).json({ message: "Not
authentication" });
};
```

Appendix B: Screenshots and System Outputs

This section presents additional screenshots and system outputs of the Employee Record Management System. These visuals demonstrate the working of the application, database storage, and backend execution, providing a clear understanding of the system functionality.

B.1 MongoDB Database View

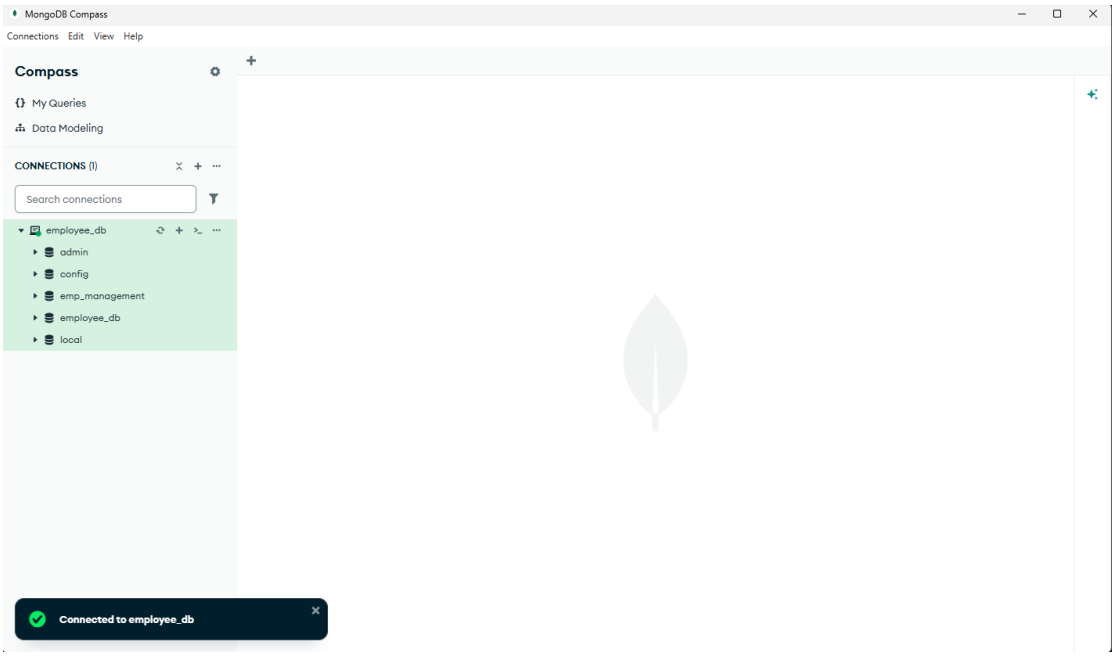

Figure 11.1.1: *MongoDB Database View*

This image shows the MongoDB Compass interface with a successful connection to the *employee_db* database.

B.2: Employee Records in Database

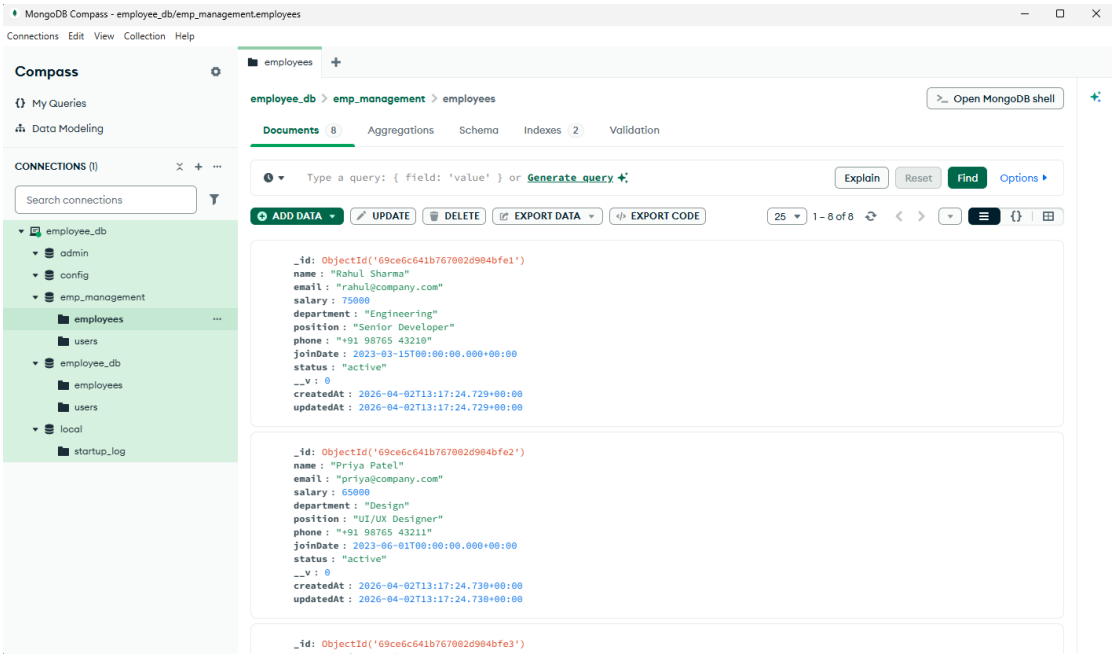

Figure 11.2.1: *Employee Records in MongoDB*

This screenshot shows the employee records stored in MongoDB Compass. It includes details such as name, email, department, salary, position, and status. The data is stored in document format within the employees collection.

B.3: Backend Server Running

Figure 11.3.1: *Backend Server Running with MongoDB Connection*

This screenshot shows the backend server running successfully in the terminal. It indicates that the server is active on port 5000 and is successfully connected to MongoDB, confirming proper backend setup and database connectivity.

B.4: Frontend Application

This section shows the frontend interface of the Employee Record Management System. The application is developed using React and provides a user-friendly interface for managing employee records, including features such as adding, updating, deleting, searching, and filtering employees.

Appendix C: API Endpoints

This section lists the RESTful API endpoints used in the Employee Record Management System for performing various operations such as authentication and employee data management.

C.1 Employee APIs

```
GET     /api/employees
```

Retrieves all employee records from the database. It also supports search by name and filtering by department using query parameters.

```
POST     /api/employees
```

Adds a new employee record to the database. The request includes details such as name, email, username, password, department, position, salary, and status.

```
PUT     /api/employees/:id
```

Updates the details of an existing employee using the employee ID. It allows modification of fields such as name, email, department, position, salary, and status.

```
DELETE  /api/employees/:id
```

Deletes an employee record from the database using the employee ID.

C.2 Authentication APIs

```
POST    /api/auth/login
```

Authenticates user credentials (admin or employee) and creates a session. Returns user role and username upon successful login

```
POST    /api/auth/logout
```

Logs out the user by destroying the session and clearing the session cookie.

```
GET     /api/auth/me
```

Checks if a user is currently logged in and returns session details such as username and role.

Appendix D: Project Structure

This section describes the folder structure of the Employee Record Management System stored in the local system.

```
D:\codes\Mern_Project\
│
├── backend\
│   ├── models\
│   │   ├── Employee.js
│   │   └── User.js
│   │
│   ├── routes\
│   │   ├── employeeRoutes.js
│   │   └── authRoutes.js
│   │
│   ├── .env
│   ├── server.js
│   └── package.json
│
├── frontend\
│   ├── src\
│   │   ├── components\
│   │   │   ├── Login.jsx
│   │   │   ├── Dashboard.jsx
│   │   │   ├── EmployeeForm.jsx
│   │   │   └── EmployeeList.jsx
│   │   │
│   │   ├── App.jsx
│   │   └── index.css
```

```
│   │
│   ├── index.html
│   └── package.json
```

Appendix E: Installation and Execution Steps

This section describes the steps required to install and run the Employee Record Management System on a local machine.

E.1 Prerequisites

- Node.js installed on the system
- MongoDB installed and running
- MongoDB Compass (optional, for database visualization)
- Code editor such as VS Code

E.2 Steps to Run the Project

1. Open terminal and navigate to the project directory:

   ```
   cd D:\codes\Mern_Project
   ```

2. Navigate to the backend folder:

   ```
   cd backend
   ```

3. Install backend dependencies:

   ```
   npm install
   ```

4. Configure the .env file with required variables:

   ```
   MONGODB_URI=your_mongodb_connection_string
   SESSION_SECRET=your_secret_key
   ```

5. Start the backend server:

   ```
   node server.js
   ```

6. Open a new terminal and navigate to the frontend folder:

   ```
   cd D:\codes\Mern_Project\frontend
   ```

7. Install frontend dependencies:

```
npm install
```

8. Start the frontend application:

```
npm run dev
```

9. Open the browser and visit:

```
http://localhost:5173
```

10. Login using default admin credentials:

    Username: admin

    Password: admin123

By:

24B11AI019

24B11AI044

24B11AI135

24B11AI146